



# Deployable Indoor-Localisation System

---

**Xanthe Cassar**  
**Ho Sum Leung**  
**Marshall Poon**

**Supervisors:**

Dr. Airlie Chapman  
Dr. Eric Schoof  
Dr. Ruixin Wen

***Executive Summary:***

Ultra-wideband (UWB) is an emerging technology and radio communication method that uses a wide frequency spectrum. UWB sensors can be implemented to determine the position of particular sensors in a network. UWB sensors transmit and receive messages between each other, and as UWB sensors are highly precise, their time-of-flight (ToF) can be measured, and subsequently the distance. For effective communication between UWB devices, synchronised clocks are essential, which necessitates the exploration of clock synchronisation algorithms. As there multiple devices and transmitting simultaneously, it is also necessity to avoid message collisions. Time-division Multiple Access (TDMA) is a scheduling algorithm to divide specific time slots for devices to transmit messages only. Overall, this report identifies and presents this proposed UWB solution to addressing indoor localisation needs, specifically for autonomous robots and vehicles in warehousing and logistics. Currently, there exists localisation systems for autonomous robots in industry that have multiple limitations and gaps. This report also documents the relevant literature and explores the scalability and accuracy of the proposed system.

# Contents

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>1</b> | <b>Glossary</b>                                | <b>4</b>  |
| <b>2</b> | <b>Introduction</b>                            | <b>4</b>  |
| <b>3</b> | <b>Background</b>                              | <b>5</b>  |
| 3.1      | Ultra-Wide Band (UWB) Sensors . . . . .        | 5         |
| 3.2      | Predecessor . . . . .                          | 5         |
| <b>4</b> | <b>Literature Review</b>                       | <b>6</b>  |
| 4.1      | Current Localisation Methods . . . . .         | 6         |
| 4.1.1    | LiDAR . . . . .                                | 6         |
| 4.1.2    | QR Code Detection . . . . .                    | 6         |
| 4.1.3    | Wireless Technology . . . . .                  | 7         |
| 4.2      | Two Way Ranging (TWR) . . . . .                | 7         |
| 4.3      | Time-Division Multiple Access (TDMA) . . . . . | 7         |
| 4.4      | Clock Synchronisation . . . . .                | 8         |
| <b>5</b> | <b>Methodology</b>                             | <b>9</b>  |
| 5.1      | Proposed Scaled Module design . . . . .        | 9         |
| 5.1.1    | Selection of Micro-controller . . . . .        | 9         |
| 5.1.2    | PCB design . . . . .                           | 10        |
| 5.1.3    | Power Supply . . . . .                         | 11        |
| 5.2      | Clock Synchronisation Algorithm . . . . .      | 12        |
| 5.2.1    | Naïve Algorithm . . . . .                      | 12        |
| 5.2.2    | Final algorithm . . . . .                      | 12        |
| 5.3      | ALOHA Protocol . . . . .                       | 13        |
| 5.4      | TWR Method . . . . .                           | 13        |
| 5.5      | TDMA Algorithm . . . . .                       | 15        |
| 5.5.1    | Algorithm . . . . .                            | 15        |
| 5.6      | Localisation Algorithm . . . . .               | 15        |
| 5.6.1    | 1D localisation . . . . .                      | 15        |
| 5.6.2    | 2D localisation . . . . .                      | 16        |
| 5.7      | Final Implementation . . . . .                 | 17        |
| <b>6</b> | <b>Results and Analysis</b>                    | <b>17</b> |
| 6.1      | Clock Synchronisation . . . . .                | 17        |
| 6.1.1    | Accuracy . . . . .                             | 17        |
| 6.1.2    | Efficiency . . . . .                           | 17        |
| 6.1.3    | Scalability . . . . .                          | 17        |
| 6.2      | Localisation . . . . .                         | 17        |
| 6.2.1    | 1D localisation . . . . .                      | 17        |
| 6.2.2    | 2D localisation . . . . .                      | 19        |
| 6.3      | TDMA . . . . .                                 | 20        |
| 6.3.1    | Scalability . . . . .                          | 21        |
| 6.3.2    | Area Coverage . . . . .                        | 22        |

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| <b>7</b> | <b>Discussion</b>                                    | <b>22</b> |
| 7.1      | Limitations of Design . . . . .                      | 22        |
| 7.2      | Future Work . . . . .                                | 23        |
| 7.2.1    | Unrestricted 2D localisation . . . . .               | 23        |
| 7.2.2    | 3D localisation . . . . .                            | 24        |
| 7.2.3    | Increasing Area Coverage . . . . .                   | 25        |
| 7.2.4    | Dynamic TDMA . . . . .                               | 26        |
| 7.3      | Integration between DWM3001CDK and ESP32S3 . . . . . | 27        |
| <b>8</b> | <b>Conclusions and Recommendations</b>               | <b>27</b> |
| 8.1      | Future Recommendations . . . . .                     | 28        |
| <b>9</b> | <b>Acknowledgements</b>                              | <b>28</b> |

# 1 Glossary

**UWB** - Ultra-Wide Band. A wireless communication protocol that allows devices to exchange data with one another via radio waves. Main feature being it occupies a wide band of frequencies, making it very robust against signal interference.

**TDMA** - Time-Division Multiple Access. A channel access method commonly used in digital cellular telephone and radio communication. It allows multiple devices to share the same frequency channel for communication by allocating time slots to each device.

**GPS** - Global Positioning System. A satellite-based radio navigation system that provides users around the globe with location and time information.

**TWR** - Two Way Ranging. A method that uses time-of-flight between devices and delays between message transmissions and receptions to calculate the distance between those devices.

**Node** - General term for calling the individual UWB modules present in the system.

**Anchor** - UWB modules in the system that are fixed and stationary. They have known distances from other anchors.

**Tag** - UWB modules in the system that mobile.

**Localisation** - Identifying the position of tags within a network of anchors

**ToF** - Time of Flight. Time taken for messages to be transmitted between devices.

**PCB** - Printed Circuit Board. A board that mechanically supports and electrically connects electro

# 2 Introduction

Recently, autonomous robots have been increasingly deployed in industries such as warehousing, manufacturing, and logistics. These robots and vehicles play a critical role in automating tasks like inventory management, material transport, and order fulfillment. These vehicles range from mobile platforms and automated guided vehicles (AGVs) to robotic systems to operate with minimal human intervention, improving efficiency, reducing labor costs, and minimizing errors in operational environments. To be autonomous, these robotic vehicles require their location and position, which is currently obtained with a variety of methods.

Global Position System (GPS) is satellite-based navigation, allowing users to obtain location and real-time information from Earth. Used world-wide and despite its wide and robust capabilities, there are some limitations with GPS in specific environments. In outdoor environments, GPS provides reliable location data, enabling robots to navigate vast spaces. GPS, however, has limited utility indoors due to weak or nonexistent signal availability. Large indoor environments like warehouses, manufacturing plants, and distribution centers lack sufficient satellite signal coverage, rendering GPS ineffective for precise positioning. This necessitates the use of alternative localisation technologies.

As such, this project aims to address situations where GPS is unavailable or weak, particularly indoors. The proposed system should be able to provide accurate measurements and real-time positioning as a substitute for GPS. Ultra-Wide Band (UWB) sensors allow inter-communication with each other by sending and receiving data packets. By utilising clock synchronisation between UWB sensors, it will be possible to determine the positions of the respective sensors relative to each other. The specific implementation that this addresses is the Time Division Multiple Access (TDMA) algorithm. TDMA will be used to allocate specific time slots for transmitting data between different sensors, and Time-of-Flight (ToF) measurement will then be used to calculate position of the devices.

In an indoor context, this deployable localisation system could be implemented in autonomous vehicles in settings such as warehouses and factories, to ensure that they have accurate location and timing measurements, as GPS would be unreliable and inaccurate. This report outlines the methodology implemented to synchronise the UWB sensors' clocks, as well as the algorithm used to calculate position.

This project aimed to be a continuation of Wen et al. (2019)'s project where they had developed the initial implementation of a UWB positioning system, intended for drone positioning. A key difference however, is that this project aims to utilise dynamic scheduling of messages with TDMA, as well as prove further scalability of the system, for indoor robots.

## **3 Background**

### **3.1 Ultra-Wide Band (UWB) Sensors**

UWB sensors are frequency-based devices that emit pulses which are swept over a large frequency band, typically ranging from 3.1 GHz and 10.6 GHz Liwari, 2024. UWB signals consist of low-power pulses, which allows precise time-of-flight ToF measurements and robustness to interference from external signals. A significant UWB manufacturer is Qorvo Inc. Formerly known as Decawave, Qorvo develops UWB and other communication sensors, devices and frameworks. UWB is a low-cost and high-accuracy alternative to current localisation techniques. While there exists applications that utilise basic UWB positioning, there are few that are dynamic, scaleable and specifically applied to autonomous robots. Currently, indoor robots utilise multiple indoor positioning techniques. However, these methods often have limitations or disadvantages when implementing in practice and in industry.

With the prevalence of highly accurate systems such as GPS, outdoor positioning is easily achieved. However, there are environments where GPS has poor reception or weak signals. One such example is indoor environments. As such, alternative methods for localisation indoors must be explored.

### **3.2 Predecessor**

Wen et al. (2019) developed an implementation of a deployable UWB localisation system intended for unmanned aerial vehicles (UAVs). The implementation was completed using an Arduino UNO, with a custom build PCB shield. The UWB sensor used was Decawave's DWM1000 Module. The PCB was designed to ensure Serial Peripheral Interface (SPI) communication between the micro-controller, and the DWM1000 Module. Initial testing and application for this project utilised these

modules for foundational prototyping.

## 4 Literature Review

### 4.1 Current Localisation Methods

#### 4.1.1 LiDAR

One possible method of indoor localisation that is widely used is vision-based systems such as LiDAR. LiDAR, or Light Detection and Ranging, is a remote sensing technology that uses light or laser pulses to measure distances to objects via the reflection of these pulses. Dardavesis et al. (2023) implemented LiDAR sensors and point clouds to achieve localisation. Point clouds are a collection of data points in a 3-dimensional space that represent objects and environments. For reference data, Dardavesis et al. (2023) used LiDAR sensors in conjunction with existing floor plans and CAD files to create and capture point clouds of ceilings in a large indoor room. Users would then capture similar data with LiDAR while they move within the indoor space, and is uploaded to a central system. Image and feature matching algorithms such as Scale Invariant Feature Transform (SIFT), were then used to compare images and thus localise the user. This method primarily face challenges with scalability and user mobility. For an increased number of users, the LiDAR and point cloud system would struggle with multiple users tracked simultaneously.

LiDAR can also be used with algorithms such as Simultaneous Localisation and Mapping (SLAM). SLAM is a core technology used in autonomous robots to enable navigation in unknown and dynamic environments. SLAM involves creating a map of the robot’s surroundings while simultaneously estimating the robot’s position within that map. While LiDAR systems could be robust for autonomous robots, it is also highly expensive. Vision-based systems are also prone to external disturbances and factors such as lighting and obstructions.

#### 4.1.2 QR Code Detection

Currently in the automation industry specifically, indoor robots in warehousing utilise Quick Response (QR) code detection for positioning. These QR codes are scattered along walls and other surfaces, to be seen by the robot. The mobile robot utilises vision-based sensors, similar to LiDAR, to detect these QR codes that contain identification data and its global coordinates (Bach et al., 2023). This data on the robot then undergoes transformations of camera co-ordinates to calculate distance. Bach et al. (2023) also implements an Extended Kalman Filter and sensor fusion with an inertial measurement unit (IMU) to achieve estimation and further accuracy. This overall method has several limitations and constraints. The use of QR code localisation also has limited scalability, as new QR codes need to be added to the environment at a suitable amount. QR code localisation alone also is not robust and accurate enough, thus why other methods are combined such as Kalman Filters (Bach et al., 2023). Similar to the LiDAR systems, QR codes can also be face issues where physical barriers such as walls and objects, can obstruct the line of sight of the QR, which is critical for this localisation technique.

### 4.1.3 Wireless Technology

Besides UWB, Wi-Fi and Bluetooth localisation systems are also capable of determining locations wirelessly. Bluetooth-based localisation, particularly Bluetooth Low Energy (BLE), has been adopted for navigation due to its low power consumption and compatibility with most mobile and embedded systems. BLE localisation systems often use received signal strength indication (RSSI) measurements to estimate the proximity between BLE beacons and devices. Despite its similarity with UWB, a BLE system has lower positioning accuracy and higher latency. Bluetooth signals can also degrade significantly in environments with many obstacles, leading to inconsistencies in localisation performance.

Table 1: Comparison of UWB, BLE, and Wi-Fi Parameters, \* denotes optimal conditions (Inpixon, 2024)

| Feature            | BLE                             | Wi-Fi                                                  | UWB                            |
|--------------------|---------------------------------|--------------------------------------------------------|--------------------------------|
| Location Accuracy* | < 5 m                           | <10 m                                                  | 10-50 cm                       |
| Range*             | Optimal: 0-25 m                 | Up to 100 m                                            | Optimal: 0-50 m<br>Up to 500 m |
| Latency*           | Typically 3-5 s to get location | Typically 3-5 s to get location                        | < 1 ms to get location         |
| Power Consumption  | Very low                        | Moderate                                               | Low                            |
| Cost               | Medium cost                     | High cost (Low cost with existing Wi-Fi access points) | Medium Cost                    |
| Frequencies        | 2.4 GHz                         | 2.4, 5 GHz                                             | 3.1 – 10.6 GHz                 |
| Data Rate          | Up to 2 Mbps                    | Up to 1 Gbps                                           | Up to 27 Mbps                  |

## 4.2 Two Way Ranging (TWR)

For accurate determination of coordinates and positioning, the distance measurements that precedes it must be as accurate as possible. Two-Way Ranging (TWR) is a technique for finding the time of flight (ToF) between devices Neirynck et al., 2016. By using set delays between transmissions and reception of those messages and embedding timestamps in those messages, the time of flight (ToF) could be found. Speed equals distance divided by time, multiplying the ToF by the speed of light gives the distance between two modules communicating via radio pulses.

## 4.3 Time-Division Multiple Access (TDMA)

TDMA is a multiple access scheme used in radio communication, mainly used for cellular mobile systems and broadband satellite. In TDMA implementation, the available channel bandwidth is accessed by every user of the network, but each user takes turns accessing the channel in time slots (Grami, 2015). In these time slots, only one certain user is allowed to transmit and communicate. This ensures that there are no overlapping transmissions between users at the same time.

In the context of a UWB localisation system, TDMA will be used to allocate time slots for respective modules to transmit messages. There are usually two types of conflicts for transmission, primary

conflict and secondary conflict. Primary conflicts occur when a node transmits and receives at the same time slot, or receives more than one transmission intended for it (Ergen and Varaiya, 2009). Secondary conflicts develop when a node and intended receiver is within a similar transmission time intended for other nodes (Ergen and Varaiya, 2009). It is also essential for TDMA that the modules and users need to have a synchronised clocks. This will allow each module to have accurate perceptions of the allocated time slots.

Using the TDMA scheduling algorithm, it has been demonstrated that the problem is NP-complete (Ergen & Varaiya, 2009). There are multiple methods of implementing the TDMA algorithm. These include node-based approaches, which adapt classic multi-hop scheduling algorithms, while other approaches include level-based scheduling and distributed algorithm-based scheduling (Ergen & Varaiya, 2009). These algorithms have been demonstrated to successfully complete tasks such as Aerial Swarm communication (Aydin et al., 2022), and athlete tracking (Waqar et al., 2021). This establishes that TDMA is adaptable and scaleable for various applications, and also devices in a network.

#### 4.4 Clock Synchronisation

As the TDMA scheduling algorithm will be dependent on the fact that all nodes of the system operate on the same clock, the first step to implementing the TDMA algorithm will be to implement a clock synchronisation algorithm. Sundararaman et al. (2005) discusses and observes multiple clock synchronisation methods. The remote clock reading method implements timestamps that are sent with transmissions to calculate a time difference between nodes (Sundararaman et al., 2005). Another example protocol, is the time transmission method, where a central node is set at having the ‘correct’ time and each other node estimates the central node’s time (Sundararaman et al., 2005). Lastly, the offset delay method involves structuring the network into multiple layers, with each layer maintaining its own time estimation. In this approach, the set-valued estimation method is applied, where the relative time relationship between nodes is modeled as a linear equation, allowing for precise clock correction across the network (Sundararaman et al., 2005).

When observing research in appropriate clock synchronisation algorithms for UWB sensors, the main two algorithms implemented are either sender-receiver synchronisation (SRS), or receiver-receiver synchronisation (RRS). SRS uses two-way messaging to synchronise the receivers with the sender, while RRS broadcasts a mass message and then compares the receiving time between the anchors and tags in the network. There are flaws with both of these algorithms however, as they require high power consumption. Other methods such as Pairwise Broadcasting Synchronization (PBS) require less power consumption, however, this also places more constraints on the network, as PBS requires the distances between some anchors and tags to be known (Shi et al., 2020).

One such specific RRS algorithm is the ALOHA protocol. ALOHA protocol is a random access method that allows users to transmit frames whenever they have data, and retry after a random time if there are collisions (Luo, 2024). Implementing the random ALOHA protocol allows initial synchronisation between nodes by initiating ALOHA transmissions between one another, should the device want to synchronise. If another device or node receives this ALOHA transmission, it will then attempt to synchronise their respective clocks with each other, with a clock synchronisation method. If multiple devices try to send their synchronization messages at the same time, a collision will occur, and each device will need to wait for a random time before trying again.



## 5 Methodology

### 5.1 Proposed Scaled Module design

#### 5.1.1 Selection of Micro-controller

As mentioned earlier, the initial prototyping stage utilised an Arduino UNO board with the DW1000 module. However, for the final scaleable system, a comparison and analysis of different modules was conducted to consider multiple factors. In addition, the scaleable system would employ Qorvo's newer and updated DW3001C module.

Table 2: Comparison of different micro-controllers

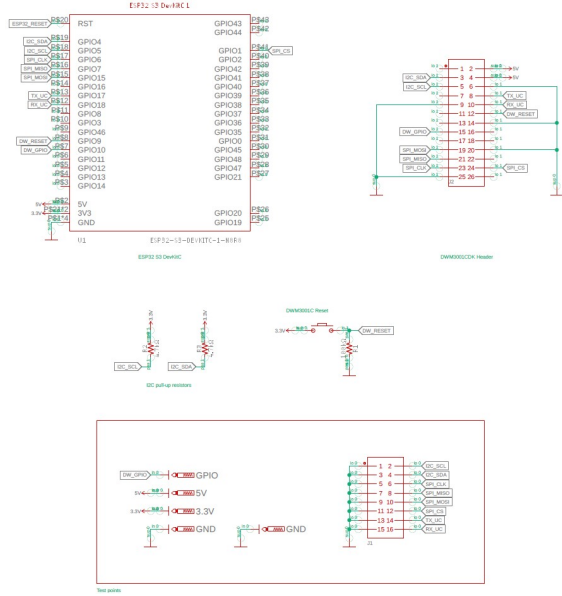
| Criteria                    | ESP32-S3 Dev Module                                                                          | Arduino UNO                                                                                  | Raspberry Pi 4                                                                                |
|-----------------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Processing Power            | Dual-core, 240 MHz<br><br>Sufficient for UWB signal processing and communication             | Single-core, 16 MHz<br><br>Limited for complex processing or multitasking                    | Quad-core, 1.5 GHz<br><br>Excellent for high-complexity tasks and multitasking                |
| Wireless Connectivity       | Built-in Wi-Fi and Bluetooth capability, ideal for wireless communication                    | No Wi-Fi capability without external module or alternative board                             | Built-in Wi-Fi and Bluetooth capability, ideal for networking and Internet-of-Things          |
| Power Consumption           | 78.32 mW                                                                                     | 1.04 W                                                                                       | 3-5 W                                                                                         |
| GPIO and Peripheral Support | 38 GPIO pins, SPI, I2C, UART, PWM, ADC, DAC                                                  | 14 GPIO pins, SPI, I2C, UART, PWM, ADC                                                       | 40 GPIO pins, SPI, I2C, UART, PWM, ADC                                                        |
| Ease of Use                 | Easy-to-use libraries, strong community support<br><br>Moderate learning curve for beginners | Very easy to use, especially for beginners<br><br>Ideal for educational purposes             | Higher learning curve<br><br>More suited to experienced developers                            |
| Cost                        | Affordable, typically around \$5-\$10                                                        | Very affordable, typically \$10-\$20<br><br>Suitable for small-scale or educational projects | More expensive, around \$35-\$75<br><br>Higher cost may not be justified for simpler projects |
| Size and Portability        | Compact, small form factor                                                                   | Compact, small form factor                                                                   | Larger form factor                                                                            |

| Criteria                | ESP32-S3 Dev Module                                                                           | Arduino UNO                                                                             | Raspberry Pi 4                                                                                                                             |
|-------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Development Flexibility | Programmable in C/C++, MicroPython, and Arduino IDE<br><br>Extensive support for IoT projects | Programmable in Arduino IDE<br><br>Limited flexibility in high-performance applications | Programmable in various languages, including Python, C/C++, and more<br><br>Capable of running a full Linux OS for development flexibility |
| Power Management        | Low-power modes, good for battery-operated projects                                           | Limited power management options                                                        | Requires external solutions for efficient power management                                                                                 |
| Community and Resources | Large and growing community, with extensive tutorials                                         | Very large community, with numerous resources and examples                              | Active community, vast range of libraries and projects                                                                                     |
| Scalability             | Scalable to small to medium IoT applications                                                  | Limited scalability due to lower processing power                                       | Highly scalable, suitable for complex projects                                                                                             |

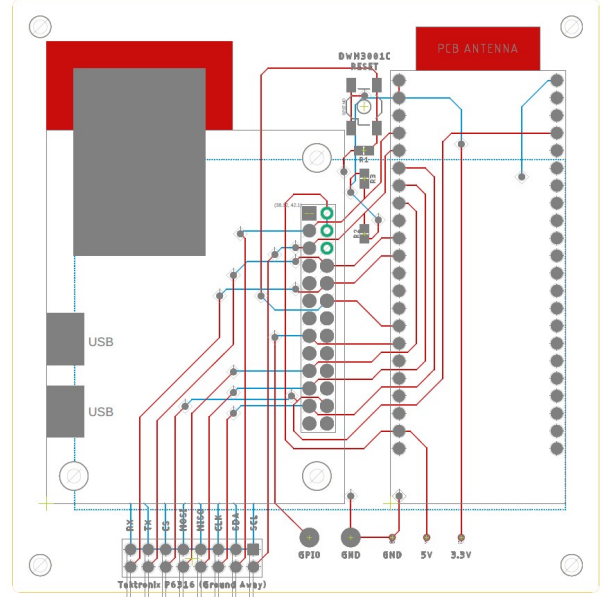
From the above table, it was determined that the ESP32S3 would be ideal for our modules as they contain the functionality and processing power required, without being unnecessarily over-specified.

### 5.1.2 PCB design

Figure 1 illustrates the PCB design of the connections between the proposed ESP32S3 and the DWM3001CDK. During development, this board was designed by our supervisor Dr. Eric Schoof. The DWM3001CDK has communication pins dedicated for a Raspberry Pi 4, which its equivalent pins are designed for the ESP32S3.



(a) ESP32 and DWM3001CDK Pinouts



(b) PCB Design

Figure 1: PCB Schematic

### 5.1.3 Power Supply

Table 3: Comparison of Power Sources for UWB Localisation Systems

| Feature               | Powerbanks                                                         | LiPo Batteries                                                            | Wired Power                                           |
|-----------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------|
| <b>Energy Density</b> | Moderate (depends on capacity, typically 3.7V Li-ion cells inside) | High (higher energy density than powerbanks due to advanced cell design)  | N/A (Unlimited supply depending on power source)      |
| <b>Portability</b>    | High (Portable and rechargeable, easy to carry)                    | Very High (Lightweight and compact, available in custom shapes and sizes) | None (Fixed to location, requires infrastructure)     |
| <b>Weight</b>         | Moderate to heavy (depending on capacity, casing, and extras)      | Very light (especially good for mobile or compact devices)                | N/A (Depends on connected device)                     |
| <b>Power Output</b>   | Moderate (Typically limited to 5V/2.4A outputs, varies by model)   | High (Capable of delivering high current for short bursts)                | Unlimited (Depends on external power source capacity) |
| <b>Cost</b>           | Affordable (depending on capacity, typically \$20-\$100)           | More expensive (especially for high-quality, custom-sized cells)          | Minimal (Electricity costs, no battery costs)         |

Table 3 shows a comparison of the power source options available for the modules. After evaluation, it was determined to power the modules with a combination of power banks (for Tags) and wired

connections (for Anchors).

## 5.2 Clock Synchronisation Algorithm

Before implementing a scheduling algorithm, it is essential that all modules within the system agree on a common time reference. Achieving this is challenging because, even if identical modules are manufactured under the same conditions and powered on simultaneously, slight variations in their clocks will occur. To address this issue, a clock synchronisation algorithm must be established prior to implementing any scheduling algorithm.

The algorithms implemented all take advantage of the fact that any message sent will be received less than a millisecond later on the receiving device at the current scale of operation. This means that the reception time of a message, for clock synchronisation purposes, can be taken as the same time the message was sent.

### 5.2.1 Naïve Algorithm

The first approach to clock synchronisation involved a simple algorithm where one module is designated as the 'master'. The master module periodically sends a synchronisation signal, or 'poll', at predefined intervals to the other modules in the system. Upon receiving the poll, each receiving module adjusts its internal clock to match the time of reception. This method ensures synchronisation across the system within each polling interval, allowing message scheduling to occur in sync. However, this algorithm has notable limitations. The most significant drawback is its dependency on a single master module: if the master becomes unavailable or disconnected, the remaining modules will gradually desynchronise, as their clocks drift apart over time.

#### Transmitter Pseudocode

```
1: zero time = current time
2: while True do
3:   if current time = 0 then
4:     Send poll message
5:   end if
6: end while
```

#### Receiver Pseudocode

```
1: while True do
2:   Listen for message
3:   if Poll message received then
4:     zero time = current time
5:   end if
6:   find adjusted time
7: end while
```

While simple, the naive algorithm ensures short-term synchronisation but suffers from single-point failure risks, leading to potential desynchronisation if the master module fails. This algorithm also requires two sets of code on different modules, one for transmission and reception, which is not ideal for our purposes.

### 5.2.2 Final algorithm

A more robust clock synchronisation algorithm improves upon the naive design by removing the dependency on a single master module. In this approach, the responsibility for sending the poll message is dynamically rotated among all modules based on a predefined sequence. Each module expects the poll message around the same time, and if its turn comes up, it takes on the role of the poll sender. This rotation ensures redundancy: if one module becomes unavailable, others can continue sending the poll message, maintaining synchronisation across the system.

#### Final Algorithm Pseudocode

```
1: sequence number = 0
```

```

2: while True do
3:   if (adjusted time = 0) & (sequence number % 3 = module id) then
4:     Send poll message
5:     sequence number++
6:   else if poll message received then
7:     zero time = current time
8:     sequence number++
9:   end if
10:  find adjusted time
11: end while

```

By distributing the task of sending the poll message, this algorithm significantly improves fault tolerance. Even if multiple modules lose connection or fail, the remaining operational modules continue to synchronise with one another, preventing de-synchronisation and ensuring that the system operates smoothly without a single point of failure.

### 5.3 ALOHA Protocol

Upon startup, each module must either integrate into an existing communication sequence or initiate a new one with any other modules that are also attempting to establish communication. To facilitate this process, the ALOHA protocol was implemented. In this protocol, a module broadcasts an "ALOHA" message at random intervals and listens for a response in the intervening periods. If a response is received, the module can then process the message and determine whether a communication sequence exists, or if it needs to establish a new sequence.

#### ALOHA Protocol Pseudocode

```

1: while No message received do
2:   Send ALOHA message
3:   Listen for random interval for response
4: end while
5: if message received then
6:   if message = ALOHA then
7:     Send response message
8:     Start poll sequence
9:   else
10:    Wait for poll message
11:   end if
12: end if

```

In the algorithm above, if the received message is another "ALOHA" message, it indicates that no existing communication sequence is in place. In response, the module will attempt to establish a new sequence by initiating the polling process. On the other hand, if a different message type is received, the module will recognise that a communication sequence is already active. In this case, it will wait for the next poll message to synchronise itself with the ongoing sequence.

### 5.4 TWR Method

The process of the TWR message exchange can be seen in Figure 2, while the ToF was given by Equation 1. However, this only determined the distance between 2 modules, i.e., the distance between 1 tag and 1 anchor. In order to localise a tag, the distances between the tag and multiple anchors had to be found for its actual location to be triangulated.

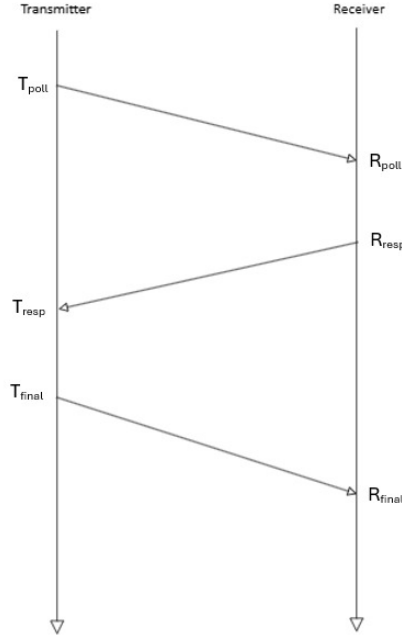


Figure 2: Message Exchange Timeline - shows which timestamps are to be recorded and sent to the Receiver such that ToF can be calculated

$$\begin{aligned}
T_{round_1} &= T_{resp} - T_{poll} \\
T_{round_2} &= R_{final} - R_{resp} \\
T_{reply_1} &= T_{final} - T_{resp} \\
T_{reply_2} &= R_{resp} - R_{poll} \\
ToF &= \frac{T_{round_1} \times T_{round_2} - T_{reply_1} \times T_{reply_2}}{T_{round_1} + T_{round_2} + T_{reply_1} + T_{reply_2}}
\end{aligned} \tag{1}$$

The pseudocode to implement the above time of flight calculations can be seen below:

#### Transmitter Pseudocode

```

1: Send Initial Poll Message
2: while True do
3:   if Receive Response then
4:     Record timestamp received  $T_{resp}$ 
5:     Set fixed delay
6:     Send final message at  $T_{final}$ 
7:   end if
8: end while

```

#### Receiver Pseudocode

```

1: while True do
2:   if Receive Initial Poll Message then
3:     Record timestamp received  $R_{poll}$ 
4:     Embed  $T_{poll}$ ,  $T_{resp}$ ,  $T_{final}$  into final
      message
5:     Set fixed delay
6:     Send return message at time  $R_{resp}$ 
7:   end if
8:   if Receive Final Message then
9:     Record timestamp received  $R_{final}$  and
      the embedded timestamps
10:    Calculate Distance
11:   end if
12: end while

```

## 5.5 TDMA Algorithm

To ensure that TWR can be completed frequently enough, while maintaining the clock synchronisation sequence, TDMA was utilised to divide up the time between polling sequences.

### 5.5.1 Algorithm

The basic TDMA algorithm acts such that it first checks if a clock sync poll message is expected, and then identifies whether the module should be sending a message or listening for the response.

```
1: while True do
2:   if (adjusted time <100) or (adjusted time >16900) then
3:     if Turn to send clock sync poll then
4:       if adjusted time <100 then
5:         Send poll
6:       else
7:         Wait for poll
8:       end if
9:     end if
10:  else if Scheduled to send message then
11:    Send message
12:  else if Expecting message then
13:    Listen for message
14:  end if
15: end while
```

While the above pseudocode demonstrates the basic concept behind TDMA, the actual implementation varied slightly. As the final system implemented consisted of two anchors and a tag, the tag was responsible for managing the TDMA schedule and initiating TWR with each of the anchors. The anchors were tasked with listening for the TWR sequence as initiated by the tag. The messages sent by the tag contained an identifier for which anchor it was attempting to communicate with. An anchor would only respond to a TWR initiation from a tag if its ID was embedded in the message's content.

## 5.6 Localisation Algorithm

It was observed that it would be too difficult to directly attempt to localise a tag. Therefore, the full localisation algorithm was divided into 4 different phases. This was developed from a simple 1-dimensional localisation to the full complex 3D space localisation.

### 5.6.1 1D localisation

The first stage was to attempt to track the position of a tag that moves away and towards an anchor in a straight line. The tag was to track its distances from the anchor during the process. Several points along the line were selected where the distance readings from the tag would be compared against real life measurements to check for discrepancies. Since the tag was just moving along a straight line, no additional mathematics outside of distance calculations from TWR was needed.

After comparing distance readings against ground truth, if the difference was larger than a threshold of about 20cm, changes were made to the code-defined antenna delays between the reception and

transmission of messages involved in the TWR process. This is because those delays directly affect timestamp values used in ToF calculations in TWR, which subsequently induce changes in distance readings. Different delays were tested until discrepancies fall below 10cm.

### 5.6.2 2D localisation

With 1D localisation achieved, the next step is to be able to localise a tag on a 2D plane. 2D localisation cannot be achieved with only 1 anchor as there is not enough information to be used for determining the coordinates of a tag with just 1 distance measurement with 1 anchor. At least 2 anchors are required to communicate with a tag to form a triangle. Then using triangulation, the position of the tag can be determined using trigonometric formulas.

Using the scheduled timeline created with TDMA, the tag would exchange messages with specific anchors in their allocated time slots such that the respective anchors could find the distances between them and the tag. These distance readings were then fed to a computer via serial for further processing. Since anchors were to be stationary nodes meant for being reference points in a coordinate system, their locations were assumed to be known and the distance between them was assumed to be fixed.

With that, all 3 edges of a triangle were determined. With the usage of the Law of Cosines, all angles within that triangle can be determined. Setting 1 of the anchors as the origin point and finding the angle (associated with it), the x and y coordinates of the tag can be determined respectively with cosine and sine relations with the edge between the origin anchor and the tag.

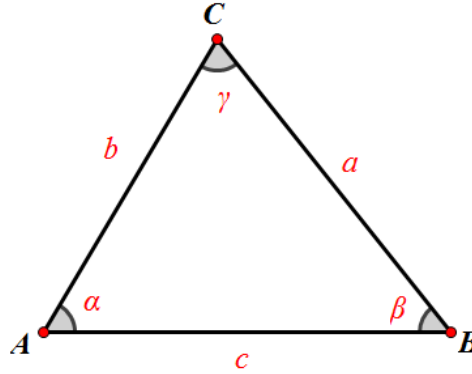


Figure 3: Triangle formed by tag C and anchor A and B

Referring to Figure 3 as an example formation of the modules. Distance readings between tag C and anchors A and B would give edge lengths b and a respectively. The distance between anchors A and B would be fixed, which can be measured and is represented by edge length c. Setting for example anchor A as the origin point, then the coordinates of tag C would turn out to be  $(b\cos(\alpha), b\sin(\alpha))$  where  $\cos(\alpha)$  and  $\sin(\alpha)$  can be found using Equations 2 and 3.

$$\cos(\alpha) = \frac{b^2 + c^2 - a^2}{2bc} \quad (2)$$

$$\sin(\alpha) = \sqrt{1 - \cos(\alpha)^2} \quad (3)$$



## 5.7 Final Implementation

Piecing together the above algorithms, any module turned on would first enter into an ALOHA sequence until it established clock synchronisation. Once clock synchronisation was established, it would then enter into the TDMA sequence where, if the module is a tag it would initiate TWR with each anchor in an alternating pattern or, if the module was an anchor, it would listen for TWR initiation from the Tag. Once the Tag has enough information from its TWR sequencing, it would then calculate its position using the localisation algorithm.

## 6 Results and Analysis

### 6.1 Clock Synchronisation

#### 6.1.1 Accuracy

The final clock synchronisation algorithm implemented capitalised on the fact that any signal sent is received in much less than 1ms, even when operated at the maximum range. This means that by taking the poll reception and send times as the 'zero time', clock synchronisation can be achieved with an accuracy of 1ms. This was determined by comparing the reported time of each module with the timestamps shown on a centralised computer.

#### 6.1.2 Efficiency

It was chosen to send a clock synchronisation poll at the start of every clock reset cycle. Since the clock reset cycle on the UWB centre is approximately 17 seconds, this means that a clock synchronisation poll was sent at the same interval.

At this clock synchronisation rate, it takes a module a maximum of 17 seconds to receive a clock synchronisation poll message from startup to be able to sync its own clock and join the TDMA schedule.

#### 6.1.3 Scalability

The clock synchronisation algorithm implemented requires any module wishing to join the synchronisation to be within range of any other module, however as any number of modules can hear a message being sent, there is functionally no limit to the number of modules that could join the synchronisation.

### 6.2 Localisation

#### 6.2.1 1D localisation

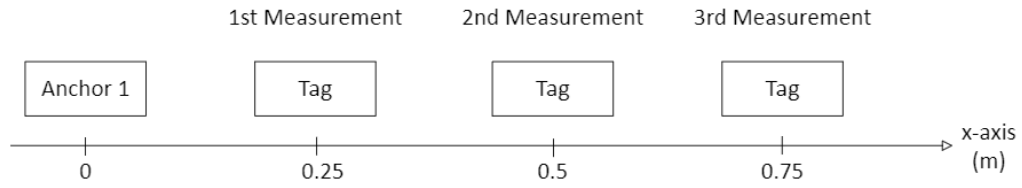
An iterative process was taken to test the accuracy of distance readings with different antenna delays. As shown in Table 4, different delays were tested at random until a certain pattern can be found and finally reaching a delay value that gives a minimal amount of error.

Table 4: Table containing the tested antenna delays

| Iteration | Antenna Delay<br>( $\mu$ s) | True Measurement<br>(m) | Distance<br>Readings (m) | $\pm$ Error<br>(m) |
|-----------|-----------------------------|-------------------------|--------------------------|--------------------|
|           | 16419                       | 0.25                    | 0.64                     | +0.39              |

| Iteration | Antenna Delay ( $\mu\text{s}$ ) | True Measurement (m) | Distance Readings (m) | $\pm\text{Error}$ (m) |
|-----------|---------------------------------|----------------------|-----------------------|-----------------------|
|           | 16419                           | 0.5                  | 0.9                   | +0.4                  |
|           | 16419                           | 0.75                 | 1.14                  | +0.39                 |
| 2         | 16500                           | 0.25                 | overflow              | N/A                   |
|           | 16500                           | 0.5                  | 0.2                   | -0.3                  |
|           | 16500                           | 0.75                 | 0.41                  | -0.34                 |
| 3         | 16400                           | 0.25                 | 0.8                   | +0.55                 |
|           | 16400                           | 0.5                  | 1.13                  | +0.63                 |
|           | 16400                           | 0.75                 | 1.32                  | +0.57                 |
| 4         | 16440                           | 0.25                 | 0.5                   | +0.25                 |
|           | 16440                           | 0.5                  | 0.7                   | +0.2                  |
|           | 16440                           | 0.75                 | 0.83                  | +0.08                 |
| 5         | 16453                           | 0.25                 | 0.38                  | +0.13                 |
|           | 16453                           | 0.5                  | 0.57                  | +0.07                 |
|           | 16453                           | 0.75                 | 0.7                   | -0.05                 |
| 6         | 16462                           | 0.25                 | 0.27                  | +0.02                 |
|           | 16462                           | 0.5                  | 0.51                  | +0.01                 |
|           | 16462                           | 0.75                 | 0.7                   | -0.05                 |

1st iteration was the original antenna delay and distance readings are consistently around 0.4m larger than the actual distance. The 2nd and 3rd iterations revealed that the ideal antenna delay should be larger than original but not beyond 16500 $\mu\text{s}$ . After the 6th iteration of testing different antenna delays, it was decided that an error of  $\pm 0.1\text{m}$  consistently was small enough and the readings are of satisfactory accuracy. As such, an antenna delay of 16462 $\mu\text{s}$  was set to be the new standard and remains in use for the remainder of the project. One thing to note is the 1st measurement in the 2nd iteration. It was observed that an antenna delay of 16500 $\mu\text{s}$  was giving distance readings that were generally about 0.3m smaller than the actual distance. Therefore, when the actual distance itself is less than 0.3m, the distance readings would overflow due to being a negative number. Figure 4 shows the set-up used for testing different antenna delays.



(a) Theoretical Experimental Set Up



(b) Actual Experimental Set Up

Figure 4: Experiment for determining antenna delays

### 6.2.2 2D localisation

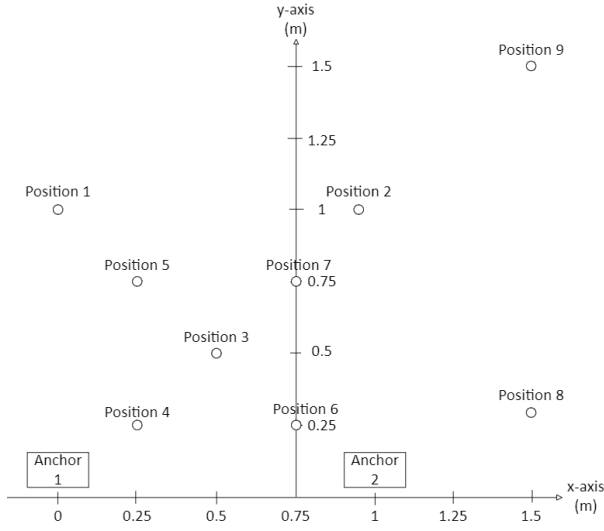
Having determined an antenna delay that provides accurate distance readings, the next task was to test how accurate the calculated coordinates of the tag are, using the methods described in the methodology section of this report. 1 anchor is designated as the origin point (0, 0) while the other is placed 1m away from it on the x-axis (1, 0). The results of the calculated coordinates of the tag is shown in Table 5.

Table 5: Table containing the tested positions in the coordinate system

| Position | Actual Coordinates<br>(x(m), y(m)) | Calculated Coordinates<br>(x(m), y(m)) | $\pm$ Error (m) |
|----------|------------------------------------|----------------------------------------|-----------------|
| 1        | (0, 1)                             | (0.01, 1.02)                           | (+0.01, +0.02)  |
| 2        | (1, 1)                             | (0.98, 1.23)                           | (-0.02, +0.23)  |
| 3        | (0.5, 0.5)                         | (0.54, 0.8)                            | (+0.04, +0.3)   |
| 4        | (0.25, 0.25)                       | (0.3, 0.6)                             | (+0.05, +0.35)  |
| 5        | (0.25, 0.75)                       | (0.1, 1.03)                            | (-0.15, +0.28)  |
| 6        | (0.75, 0.25)                       | (0.77, 0.37)                           | (+0.02, +0.12)  |
| 7        | (0.75, 0.75)                       | (0.7, 1.06)                            | (-0.05, +0.31)  |
| 8        | (1.5, 0.3)                         | (1.6, 0.56)                            | (+0.1, +0.26)   |
| 9        | (1.5, 1.5)                         | (1.9, 1.8)                             | (+0.4, +0.3)    |

From the results in Table 5, it can be seen that the calculated x-coordinates are generally very accurate, down to 10cm accuracy for most cases until the tag moves out of the enclosure of the 2 anchors. However, same cannot be said about the calculated y-coordinates. They can be seen to be consistently larger than the actual y-coordinate of the location at which the tag was placed at. This could be due to the employed method of triangulation making use of sine and cosine entities,

which could have issues with quadrants and folds. Figure 5 shows the experimental set up for testing how accurate the system's 2D localising capabilities are.



(a) Theoretical 2D Experimental Set Up



(b) Actual 2D Experimental Set Up

Figure 5: Experiment for testing the accuracy of 2D localisation

### 6.3 TDMA

For the number of devices used in the implementation described in this report, the TDMA schedule was relatively simple with only one device alternating communication with two others. With the inclusion of a time slot for the clock synchronisation, and 1000 ms left for a TWR sequence, the final TDMA schedule can be visualised as shown in Figure 6.

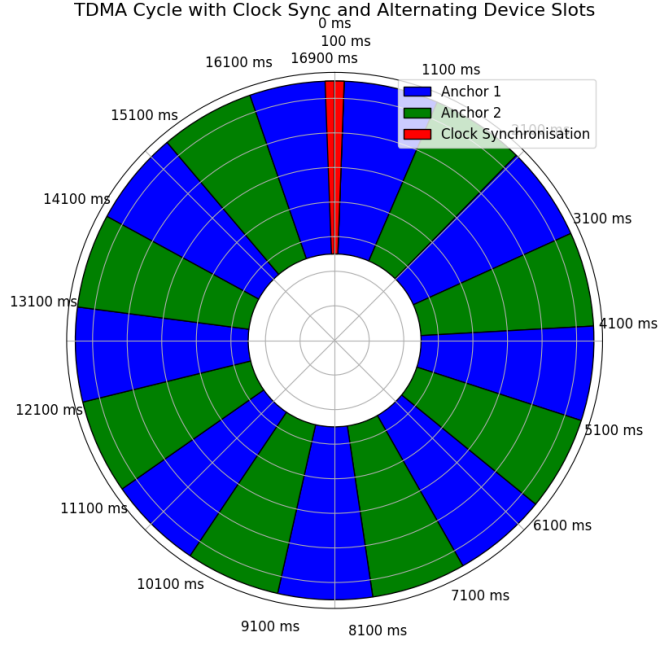


Figure 6: Visualisation of implemented TDMA schedule

### 6.3.1 Scalability

Testing of the TWR sequencing with an Arduino and UWB sensor demonstrated that it takes approximately 9ms to complete the message exchange. For a tag to be localised, it must communicate with at least three anchors in the environment for triangulation. This means the total time to localise one tag is approximately 27ms. To allow for some scheduling overhead in the scheduling and to avoid conflicts, a buffer should be added. This brings the total time to be allocated per tag for localisation to be 30ms.

The scalability of this system is heavily influenced by the accuracy requirements of the system. For higher accuracy requirements, the update rate of each tag increases, which places lower limits on the number of tags that can be localised in the system. Lower accuracy requirements allow a larger number of tags to be localised in the system.

For example, if each tag is required to update its position once per second, the number of tags the system can handle is given by:

$$\frac{1000 \text{ ms}}{30 \text{ ms/tag}} = 33 \text{ tags} \quad (4)$$

This means that for applications where it is sufficient for each tag to report its position once per second (eg. where an object may be moving slowly), 33 tags can be supported within the system.

However, if a higher level of precision is required—such as tracking fast-moving objects, where position updates are needed 10 times per second—the number of tags that can be supported in the system is reduced:

$$\frac{1000 \text{ ms}}{30 \text{ ms/tag} \times 10 \text{ updates/sec}} = 3 \text{ tags} \quad (5)$$

In this scenario, only 3 tags can be reliably localised if each requires an update of 10 Hz. This shows a trade off between the frequency of updates and the number of tags a system can handle.

### 6.3.2 Area Coverage

Given that the tested range on the UWB sensor is approximately 15m, this means that the area coverage of the developed system consisted of the intersection of the two anchor coverage areas. A visualisation of the coverage area can be shown in Figure 7.

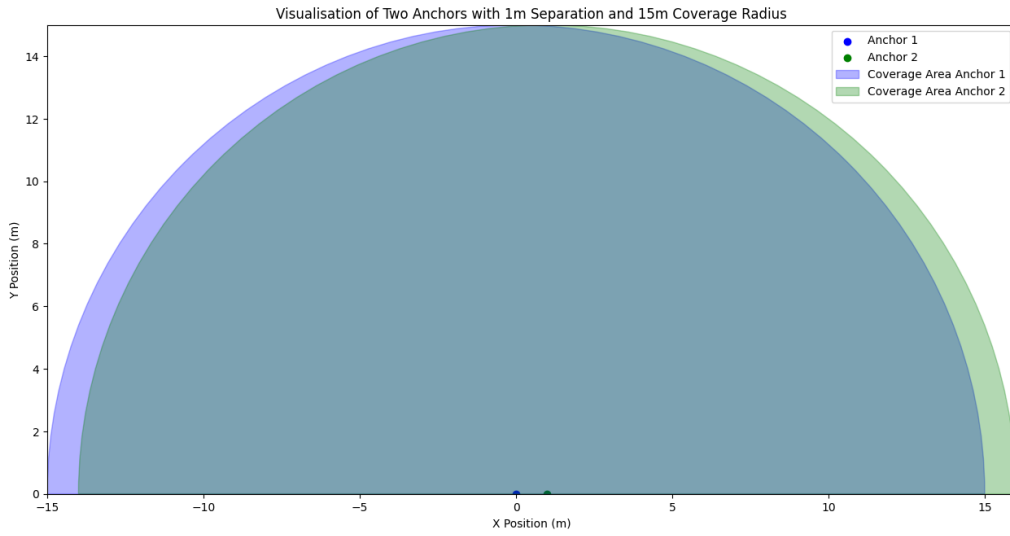


Figure 7: Visualation of coverage with two anchors

As shown above, the system implemented is able to be localised within the intersection of the shaded areas. A system with a larger number of anchors spread over a wider area would be able to localise a tag over a greater area.

## 7 Discussion

### 7.1 Limitations of Design

The system as it is now is definitely capable of tracking the position of tags moving around certain areas within the anchors' range of coverage. However, it is still subject to certain restrictions and limitations imposed by the assumptions made during the design and implementation of the system.

For example, the current version of the system has only achieved 2D localisation, meaning it can only accurately track tags that are moving on the same plane as the anchors. Once elevation is

introduced to the system, the accuracy is no longer guaranteed as the calculations do not take into account a 3rd dimension in the form of height.

As such, an assumption was made here that objects to be tracked will always be moving around the same height level, much like how land rovers or robots in warehouses would generally be moving around a relatively flat plane throughout operation and usually should not experience great upward or downward climbs in height. If there is a necessity for them to move to other levels, although the transition itself can't be tracked, after they have moved to another level, another set of anchors could then track their movement again.

Another limitation is UWB's capability to penetrate walls and objects. In most indoor settings, there are bound to be objects scattered around the area that will deny line of sight between tags and anchors. This is a problem for other wireless technologies such as WiFi or Bluetooth as they rely heavily on signal strength which gets drastically attenuated or dispersed through walls and objects, which is a weakness that UWB signals suffers less from as they measure signal travel time instead of strength.

However, UWB is still not completely impervious to obstructions. Although UWB signals can effectively travel through walls, they still experience reduced coverage range and signals may take longer to reach the intended target, which is still better than losing signal connection with the target, but remains a limitation of the system.

## **7.2 Future Work**

Given the limitations of the current localisation system, there are multiple extensions that can be adopted in the future to further expand its capabilities. Increasing the fields of application for the system and potentially making it more accurate and adaptable to different scenarios and environments.

### **7.2.1 Unrestricted 2D localisation**

The current system setup is able to determine the location of tags residing on only one half of the 2D plane separated by the axis created by the alignment between 2 anchors and is configured during the start up of the system. Meaning not the whole 2D plane is covered and thus is a restricted localisation.

A direct upgrade to the system would be to have it be able to track tags across the whole 2D plane. Although the aforementioned restriction can be overcome by the placement of more anchors that face each other, there is no guarantee that such placements are always possible. Therefore, having the system be able to operate across the whole 2D plane would make it a lot more adaptable to different environments and settings.

A possible way to achieve this upgrade is through the use of other methods in calculating the coordinates of the tag module. The current employed method uses triangulation to determine the coordinates of a varying point (tag) relative to 2 stationary points (anchors), an alternative is to use trilateration instead of triangulation. Figure 8 shows in theory how the process of determining

tag coordinates via trilateration would look like. The locations of 3 anchors would be fixed at known locations,  $d_1$ ,  $d_2$  and  $d_3$  would be the distance readings between the tag and the 3 anchors. The 3 circles each represent all the possible locations a tag could be at relative to the individual anchors given a distance reading. Solving the system of equations in 6 would give the intersection point  $(x, y)$  between the 3 circles and represents the coordinates of the tag.

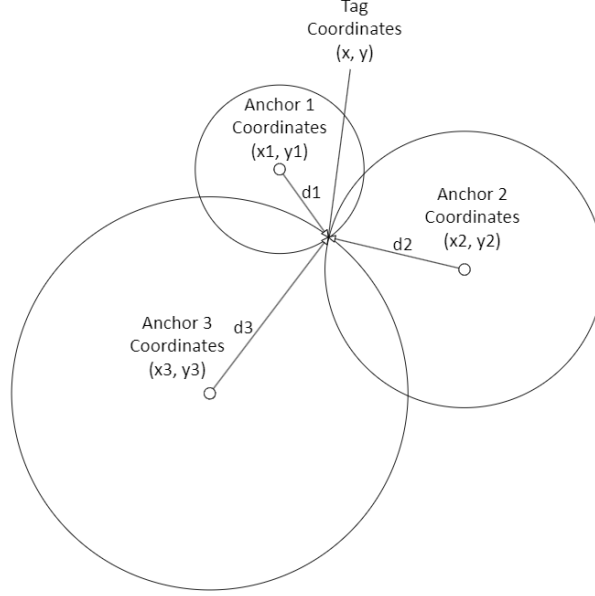


Figure 8: Circles formed by anchor 1, 2 and 3

$$\begin{aligned}
 (x - x_1)^2 + (y - y_1)^2 &= d_1^2 \\
 (x - x_2)^2 + (y - y_2)^2 &= d_2^2 \\
 (x - x_3)^2 + (y - y_3)^2 &= d_3^2
 \end{aligned} \tag{6}$$

### 7.2.2 3D localisation

Another possible extension of this system is the capability to track tags moving in 3 dimensions, which can actually be achieved also with trilateration. The only changes required would be an additional anchor since extra information is needed now to make up for the extra dimension, and instead of finding the intersecting point between 3 circles, it would be locating the intersection between 4 spheres generated by the area of coverage and distance readings of the 4 anchors. Figure 9 shows what the process if 3D trilateration would look like, and solving the system of equations in 7 would give the 3D coordinates  $(x, y, z)$  of the tag.



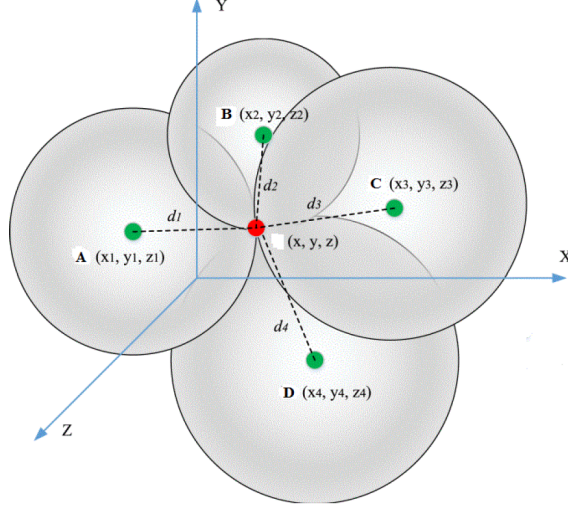


Figure 9: Spheres formed by 4 anchors (Ni et al., 2019)

$$\begin{aligned}
 (x - x_1)^2 + (y - y_1)^2 + (z - z_1)^2 &= d_1^2 \\
 (x - x_2)^2 + (y - y_2)^2 + (z - z_2)^2 &= d_2^2 \\
 (x - x_3)^2 + (y - y_3)^2 + (z - z_3)^2 &= d_3^2 \\
 (x - x_4)^2 + (y - y_4)^2 + (z - z_4)^2 &= d_4^2
 \end{aligned} \tag{7}$$

This extension opens up the application of the localisation system to tracking the position of aerial drones, which could be desirable for drone swarm applications in search and rescue and disaster relief. It would also have the capability to track land robots moving to different height levels, which the current system could still accomplish, but with 3D localisation it would require fewer anchors and fewer constraints on the placement of said anchors.

### 7.2.3 Increasing Area Coverage

The scalability of the system is not only dependent on the timing of updates but also on the geographical area covered. In a large warehouse, if the tags are physically spread out over a wide area, it is possible that their UWB messages will not interfere with each other. UWB's ability to support spatial multiplexing, combined with techniques like TDMA, means that multiple tags can occupy the same timeslot if they are far enough apart, avoiding message collisions.

In such scenarios, it is possible to schedule multiple tags within the same 30-millisecond window, further increasing the number of tags that can be localised simultaneously. The larger the warehouse or covered area, the more opportunities exist to localise multiple tags in parallel without conflicts. This allows for scaling the system up in environments where the tags are well-distributed across the available space.

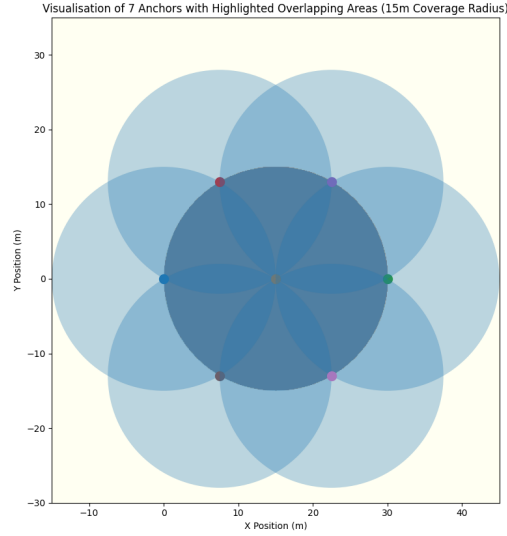


Figure 10: Visualisation of coverage with more anchors

Figure 10 demonstrates the potential coverage on a 2D plane with 7 anchors broadcasting their location. In this example, it is possible to know the unique location of a tag anywhere where two or more anchor coverage areas intersect.

When a tag is located in the centre region, it will always be able to communicate with at least 3 anchors, ensuring a unique position. Also considering that the regions where only two of the anchors communication boundaries are unique to each pair of anchors, it would also be possible to determine a unique location when a tag is located in one of these regions. There may be limitations to this however, as fewer readings from anchors could result in a less precise estimation.

Although Figure 10 highlight the communication range of the anchors, it also implies that when two tags are located in different areas of the space, specifically when they are more than 15 meters apart, they are unlikely to interfere with each other's communication. Due to the somewhat limited range of the UWB signals, this allows the tags to communicate with the anchors without 'hearing' each other's transmissions.

With careful selection of the sequencing of TWR communication, ie to ensure two tags are not attempting to talk to the same anchor at the same time, the system would be able to support multiple tags scheduled to communicate at the same time. This would allow efficient scale up of the system.

#### 7.2.4 Dynamic TDMA

The version of TDMA implemented in this project has a static schedule for the tag to communicate with the anchors. This can be extended to be more dynamic, allowing tags to enter and leave the schedule as required, as well as allowing multiple tags to communicate simultaneously if they are far enough apart.

In dynamic TDMA, it is possible to support adaptive timeslot allocation. For larger systems, the number of available time slots and their assignment to devices can change based on current network conditions, such as the number of active devices, their communication requirements, and the data traffic load. Instead of fixed slot allocation, the system can dynamically adjust who gets to transmit and when, based on demand.

In spatially separated areas, dynamic TDMA allows different devices that are far apart and unlikely to interfere with each other to reuse the same time slots. For example, two tags located in different areas of a warehouse can transmit during the same time slot because their signals won't collide.

This allows for a more efficient system, where it can allocate more slots to devices with higher communication demands, ie moving faster, and fewer slots to those with lower demands, ie moving slowly, or devices that are temporarily inactive, ie stationary. This results in better bandwidth usage and avoids wasting time slots on inactive devices.

### 7.3 Integration between DWM3001CDK and ESP32S3

The integration between the purchased DWM3001CDK and the ESP32 presented several unforeseen challenges. The purchased sensor included a development kit, which came with the UWB module itself (DW3001C) and its own onboard chip, nRF52833. Although the development kit was advertised as programmable via a Raspberry Pi, and by extension, an ESP32, further investigation revealed that the kit was not designed for direct control using an external micro-controller, as there were separate SPI communication pins between the DW3001C module, and the nRF52833, before a final separate SPI communication between this combined module, and the Raspberry Pi pins. Unfortunately, this limitation was not mentioned in the available documentation during the initial research phase.

The modules themselves have complex internal architectures, which, with further investigation, could potentially be leveraged to develop a system similar, or likely more complex than the one outlined in this report. The scalable system would have to be directly programmed onto the DWM3001CDK. The DWM3001CDK Embedded Firmware ran a complex digital systems architecture, much different to our initial development in a C/C++ environment. As such, a future implementation of our scalable system could still be possibly be implemented on the DWM3001CDK, but with a new system architecture. Our localisation system however, would likely be implemented on a micro-controller that could communicate via SPI to the DW3001C module directly. However, with the success of our dynamic clock synchronisation algorithm, in conjunction with our TDMA algorithm, our deployable localisation system would be indeed scaleable and accommodate significantly more time slots for additional nodes.

## 8 Conclusions and Recommendations

This report describes the implementation of a 2D localisation system consisting of two anchors and one tag. The system was built by integrating an Ultra-Wideband (UWB) sensor with an Arduino UNO microcontroller to communicate with the sensor to schedule, transmit and receive messages.

The Arduino was then used to process the messages and compute the Tags position. These components were interconnected via a PCB and powered with a power bank.

Upon startup, each module underwent an aloha sequence, allowing the module to either detect an existing message schedule or initiate a new one. If no schedule was detected, the module initiated the clock synchronisation process. Once synchronised, modules joined a Time Division Multiple Access (TDMA) schedule and began Two-Way Ranging (TWR) to measure distances.

Each module was assigned as either an anchor (in a fixed position) or a tag (free to move around). In this implementation, the Tag managed the TDMA schedule and initiated TWR sequences, while the Anchors responded by measuring distances based on these exchanges. The tag used these distances and the known positions of the anchors to calculate its own location through triangulation.

The clock synchronisation algorithm provided accuracy within 1 millisecond and required a maximum of 17 seconds from startup, as the clock synchronisation poll message was sent during each clock reset cycle. This scalable algorithm allowed modules within range to participate without limiting the system's capacity.

The TDMA algorithm managed communication between the tag and anchors on a fixed schedule, with an allocated time slot for reliable clock synchronisation. Through experimentation, the system achieved distance measurements with an accuracy of 0.1 meters over a range of 15 meters indoors.

After obtaining distance measurements, the tag estimated its position relative to the anchors, with a maximum uncertainty of 0.4 meters due to errors in distance measurement and quadrant calculation.

Despite the system's functionality, certain limitations were identified. The use of only two anchors restricted localisation to one-half of the 2D plane. An additional anchor would have allowed for full-plane coverage. The system also faced constraints regarding the number of tags it could track and the UWB signal's ability to penetrate walls, which should be considered for indoor applications.

## 8.1 Future Recommendations

There are several recommendations for future work to achieve beyond this report. Due to the limitation of anchor modules, this system was only able to localise on a restricted 2D plane. With access to more modules, localisation across the full 2D and into the 3D plane would also be possible.

Access to more modules would also allow the coverage area for localisation to be dramatically scaled up, as well as enable the implementation of a dynamic TDMA scheduling algorithm. This would allow the localisation of a greater number of Tags within a space.

## 9 Acknowledgements

The team would like to thank those who have supported us throughout the Capstone project. In particular, our supervisors Dr. Airlie Chapman and Dr. Eric Schoof for their contributions, mentoring and expertise. We'd also like to thank Ruixin Wen, for her continuous support and

insight on UWB localisation. Our project and success could not have been made possible without them. The team would also like to thank our families for their support throughout our degrees.

## References

- Aydm, E. E., Kara, O., Cakir, F., Cansiz, B. S., Secinti, G., & Canberk, B. (2022). Enabling Self-Organizing TDMA Scheduling for Aerial Swarms. <https://doi.org/10.1145/3539493.3539584>
- Bach, S. H., Khoi, P. B., & Yi, S. Y. (2023). Application of QR code for localization and navigation of indoor mobile robot. *IEEE Access*, 11, 28384–28390.
- Cao, Y., Chen, C., St-Onge, D., & Beltrame, G. (2021). Distributed TDMA for Mobile UWB Network Localization. *IEEE INTERNET OF THINGS JOURNAL*, 8, 13449–13464. <https://doi.org/10.1109/jiot.2021.3066243>
- Dardavesis, I., Verbree, E., & Rafiee, A. (2023). Indoor localisation and location tracking in indoor facilities based on LiDAR point clouds and images of the ceilings. *AGILE: GIScience Series*, 4, 4.
- Decawave. (2014). APS010 APPLICATION NOTE WIRELESS SENSOR NETWORKS AND THE DW1000. <https://www.qorvo.com/products/d/da007967>
- Decawave. (2016). DW1000 Device Driver Application Programming Interface (API). <https://forum.qorvo.com/uploads/short-url/jQPkxawHn3LzbJln0QqjPShspZP.pdf>
- Decawave. (2017). DW1000 Datasheet. <https://www.qorvo.com/products/d/da007946>
- Ergen, S. C., & Varaiya, P. (2009). TDMA scheduling algorithms for wireless sensor networks. 16, 985–997. <https://doi.org/10.1007/s11276-009-0183-0>
- Grami, A. (2015). *Communication Networks*. Elsevier EBooks. <https://doi.org/10.1016/b978-0-12-407682-2.00011-9>
- Inpixon. (2024). BLE (Bluetooth) Location Tracking & Positioning [Accessed: 2024-10-17].
- Liwari. (2024). *Faq about uwb* [Accessed: 18 Oct. 2024]. [https://www.iiwari.com/faq-about-uwb/?fbclid=IwZXh0bgNhZW0CMTEAAR2VDtSp8M9tzBzrkkdY\\_Po-D8PrNKmGmCYzsBsiMI1h8bEGUJraem\\_MQFXRqb3VJa5NYJLf7UWEg](https://www.iiwari.com/faq-about-uwb/?fbclid=IwZXh0bgNhZW0CMTEAAR2VDtSp8M9tzBzrkkdY_Po-D8PrNKmGmCYzsBsiMI1h8bEGUJraem_MQFXRqb3VJa5NYJLf7UWEg)
- Luo, L. (2024). Internet Technologies COMP90007 Lecture 23 Semester 2 - 2024 [Lecture Notes]. [https://canvas.lms.unimelb.edu.au/courses/184227/pages/week-4-2?module\\_item\\_id=6003555](https://canvas.lms.unimelb.edu.au/courses/184227/pages/week-4-2?module_item_id=6003555)
- Neiryneck, D., Luk, E., & McLaughlin, M. (2016). An alternative double-sided two-way ranging method. 2016 13th workshop on positioning, navigation and communications (WPNC), 1–4.
- Ni, D., Postolache, O. A., Mi, C., Zhong, M., & Wang, Y. (2019). UWB indoor positioning application based on Kalman filter and 3-D TOA localization algorithm. 2019 11th International Symposium on Advanced Topics in Electrical Engineering (ATEE), 1–6. <https://doi.org/10.1109/ATEE.2019.8724905>
- Shi, Q., Cui, X., Zhao, S., Xu, S., & Lu, M. (2020). BLAS: Broadcast Relative Localization and Clock Synchronization for Dynamic Dense Multiagent Systems. *IEEE Transactions on Aerospace and Electronic Systems*, 56, 3822–3839. <https://doi.org/10.1109/taes.2020.2979640>
- Sundararaman, B., Buy, U., & Kshemkalyani, A. D. (2005). Clock synchronization for wireless sensor networks: a survey. 3, 281–323. <https://doi.org/10.1016/j.adhoc.2005.01.002>
- Waqar, A., Ahmad, I., Habibi, D., & Phung, Q. V. (2021). Analysis of GPS and UWB positioning system for athlete tracking. 14, 100036. <https://doi.org/10.1016/j.measen.2020.100036>
- Wen, R., Taylor, S., Morphy, P., & Stewart, M. (2019). Deployable Drone Localisation System.